

Daily Problem: March 8, 2026

Problem Statement

Let

$$H = \{\langle M, w \rangle \mid M \text{ is a Turing machine that halts on input } w\}.$$

Prove that H is recursively enumerable but not recursive. That is, show that no Turing machine can decide H , even though one can semi-decide it.

Solution

We prove the statement in two parts.

First, we show that H is recursively enumerable. This means that there exists a Turing machine that accepts exactly those inputs $\langle M, w \rangle$ for which M halts on w , although it may run forever otherwise.

Second, we show that H is not recursive. This means that there is no Turing machine that always halts and correctly decides whether $\langle M, w \rangle \in H$. The proof of this part uses diagonalization.

Step 1: Showing that H is recursively enumerable

We construct a Turing machine U that works as follows on input $\langle M, w \rangle$:

1. Interpret the input as an encoding of a Turing machine M and a string w .
2. Simulate the computation of M on input w step by step.
3. If the simulation ever reaches a halting state of M , then accept.

This machine U semi-decides H .

Indeed, if $\langle M, w \rangle \in H$, then M halts on input w after finitely many steps. Therefore the simulation performed by U will eventually detect this and accept.

On the other hand, if $\langle M, w \rangle \notin H$, then M never halts on input w . In that case the simulation continues forever, so U never accepts.

Hence U accepts exactly the strings in H , and therefore H is recursively enumerable.

Step 2: Assuming for contradiction that H is recursive

Now suppose, for contradiction, that H is recursive.

Then there exists a Turing machine D that decides H . In other words, on every input $\langle M, w \rangle$, the machine D halts and behaves as follows:

$$D(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ halts on } w, \\ \text{reject} & \text{if } M \text{ does not halt on } w. \end{cases}$$

We will use D to construct a new machine that leads to a contradiction.

Step 3: Constructing the diagonal machine

Define a Turing machine N that, on input $\langle M \rangle$ (an encoding of a Turing machine M), does the following:

1. Run the decider D on input $\langle M, \langle M \rangle \rangle$.
2. If D says that M halts on input $\langle M \rangle$, then N goes into an infinite loop.
3. If D says that M does not halt on input $\langle M \rangle$, then N halts immediately.

So N is designed to do the opposite of what M does on its own encoding, at least with respect to halting.

More formally,

$$N(\langle M \rangle) = \begin{cases} \text{loop forever} & \text{if } M \text{ halts on } \langle M \rangle, \\ \text{halt} & \text{if } M \text{ does not halt on } \langle M \rangle. \end{cases}$$

This construction is valid because we assumed that D always halts and correctly answers the halting question.

Step 4: Applying N to its own encoding

Now consider what happens when we run N on its own encoding $\langle N \rangle$.

There are two possibilities.

Case 1: N halts on input $\langle N \rangle$. If N halts on $\langle N \rangle$, then by definition of D ,

$$D(\langle N, \langle N \rangle \rangle)$$

must accept, since N does halt on its own encoding.

But then, by the construction of N , when D accepts, the machine N must loop forever.

This contradicts the assumption that N halts on input $\langle N \rangle$.

Case 2: N does not halt on input $\langle N \rangle$. If N does not halt on $\langle N \rangle$, then

$$D(\langle N, \langle N \rangle \rangle)$$

must reject, since N does not halt on its own encoding.

But then, by the construction of N , when D rejects, the machine N halts immediately.

This contradicts the assumption that N does not halt on input $\langle N \rangle$.

In both cases we obtain a contradiction.

Step 5: Concluding non-recursiveness

The contradiction arose from assuming that there exists a decider D for H .

Therefore no such decider can exist. Hence H is not recursive.

Conclusion

We have shown two things:

- H is recursively enumerable, because a universal simulation can run M on w and accept if M ever halts.
- H is not recursive, because if a decider for H existed, we could build a diagonal machine N that halts on $\langle N \rangle$ if and only if it does not halt on $\langle N \rangle$, which is impossible.

Thus H is recursively enumerable but not recursive.